

The background of the entire image is a solid red color. Overlaid on this background is a large, stylized arch shape composed of numerous small, light red dots. The dots are arranged in a way that creates a sense of depth and movement, with the arch curving from the left side towards the right. In the center of this arch, the word "HUST" is written in a bold, white, sans-serif font.

HUST

ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

ONE LOVE. ONE FUTURE.



**ĐẠI HỌC
BÁCH KHOA HÀ NỘI**
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

BÁO CÁO BÀI TẬP LỚN

Học phần: IT4490 - Thiết kế và xây dựng phần mềm

Lớp: 152250

GVHD: Nguyễn Thị Thu Trang

Nhóm: 2

ONE LOVE. ONE FUTURE.

THÀNH VIÊN NHÓM

- Phạm Minh Trường 20215292
 - Payment with VnPay
- Nguyễn Hùng Cường 20215264
 - Usecase Search products + Sort products + Filter by Category
- Hoàng Nguyễn Trường Giang 20215270
 - Usecase Place order + Place Rush order
- Bùi Khánh Hoàng 20215273
 - Payment + view ordered list



NỘI DUNG

1. Giới thiệu chung
2. Kiến trúc hệ thống và thiết kế hệ thống
3. Thiết kế chi tiết
4. Phân tích thiết kế



1. Giới thiệu chung

- Dự án AIMS là hệ thống quản lý đơn hàng trực tuyến, cho phép người dùng tìm kiếm sản phẩm, đặt hàng online và thanh toán qua VNPay.
- Hệ thống này nhằm mục đích cung cấp trải nghiệm hiệu quả và thân thiện với người dùng, đồng thời có khả năng mở rộng trong tương lai cho các tính năng bổ sung.

NỘI DUNG

2. Kiến trúc hệ thống và thiết kế hệ thống

2.1. Architectural Patterns

2.2. Usecase Diagram

2.3. Activity Diagram

2.4. Sequence Diagram

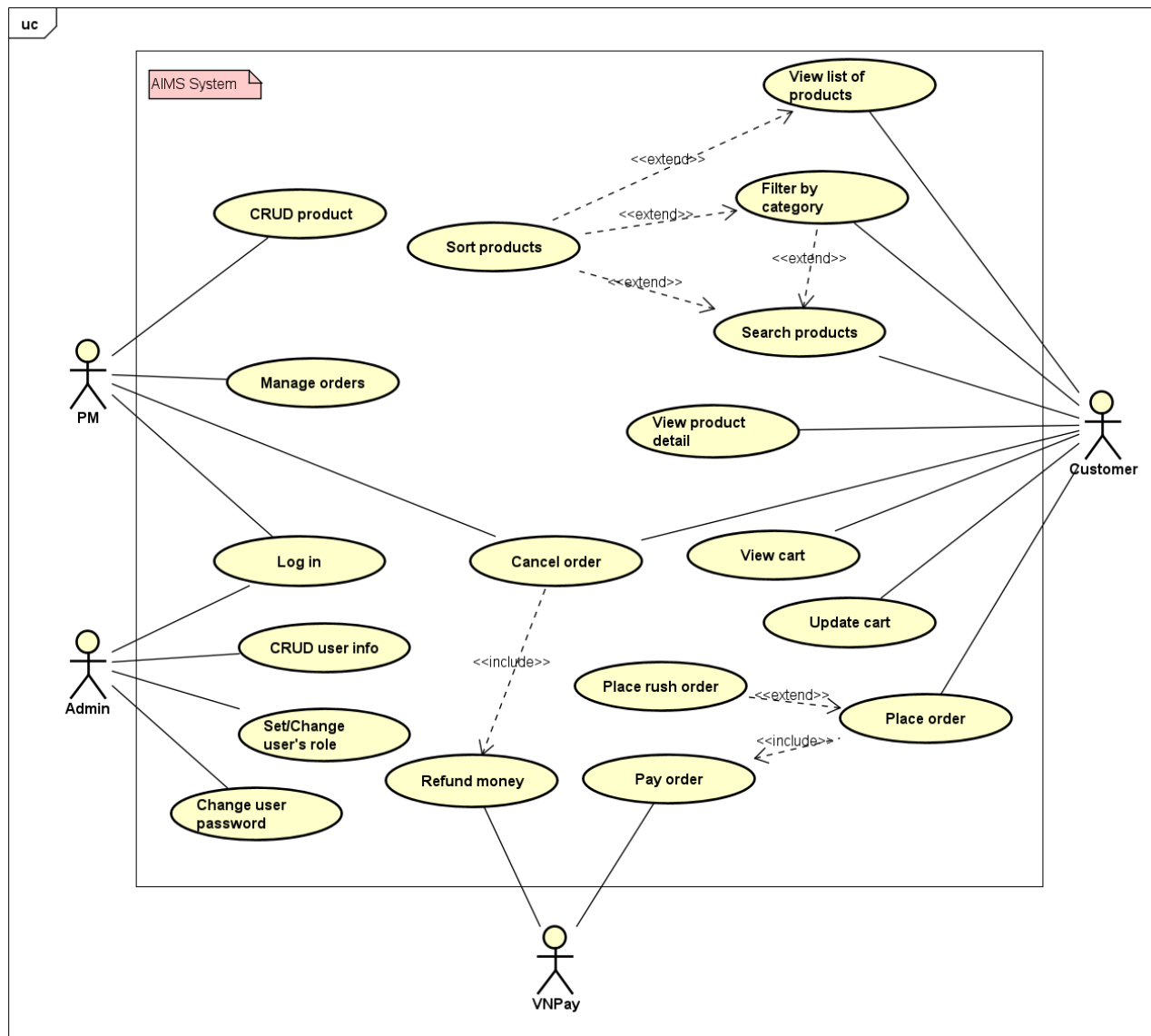
2.5. Communication Diagram

2.6. Class Diagram

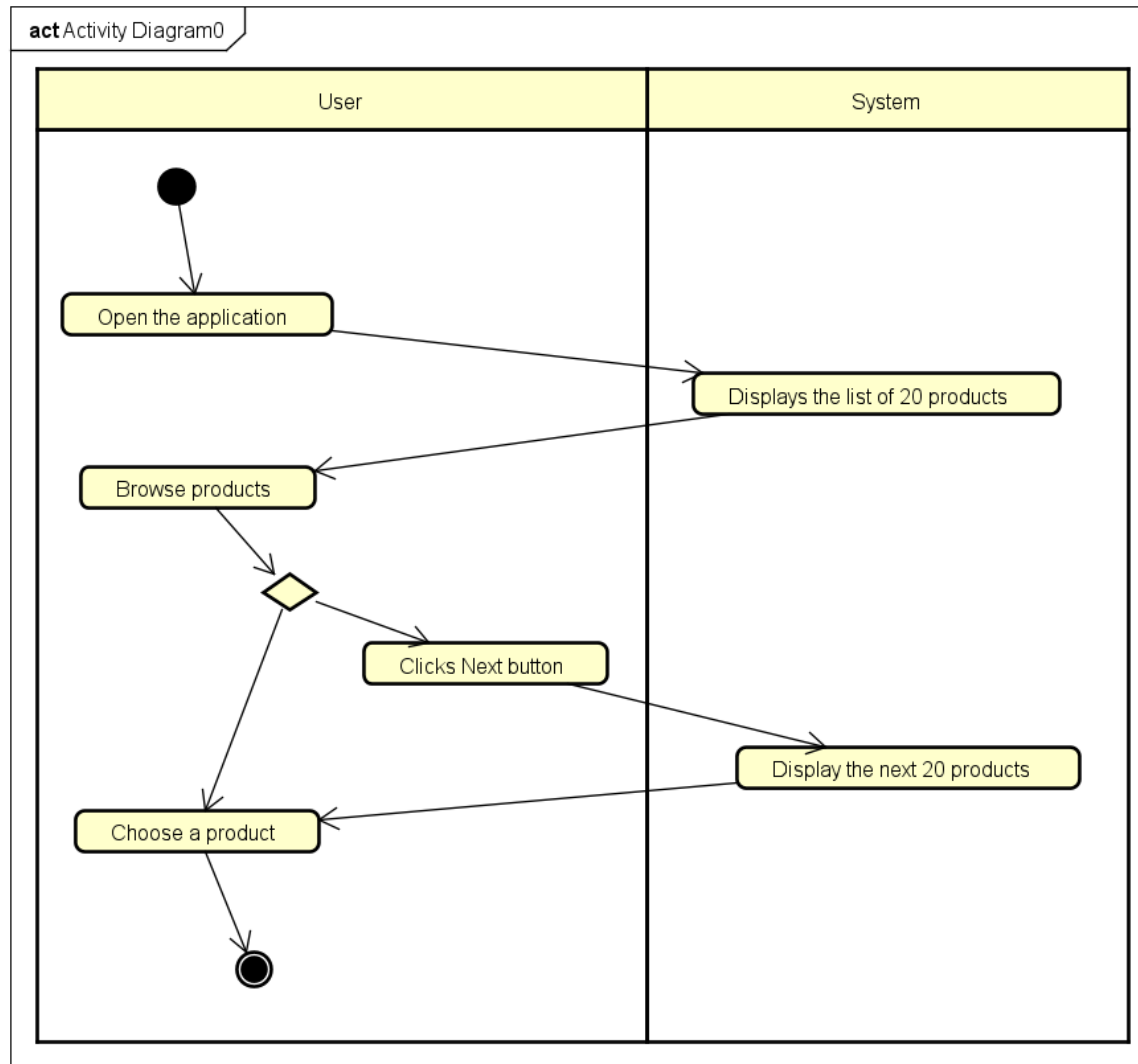
2.1. Architecture Patterns

- MVC Architecture
- Layered Architecture
- Service/Subsystem Design

2.2. Usecase diagram

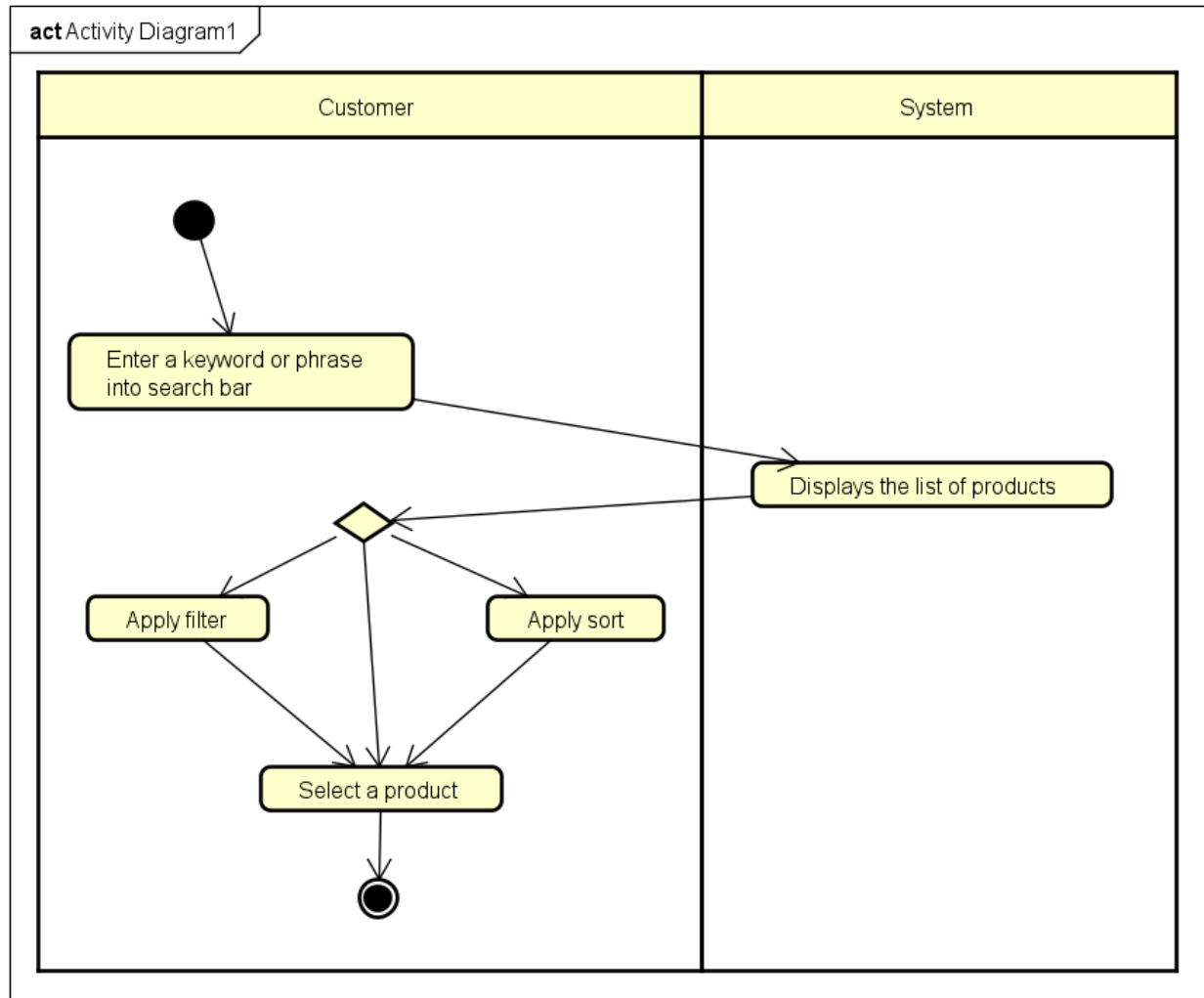


2.3. Activity Diagram



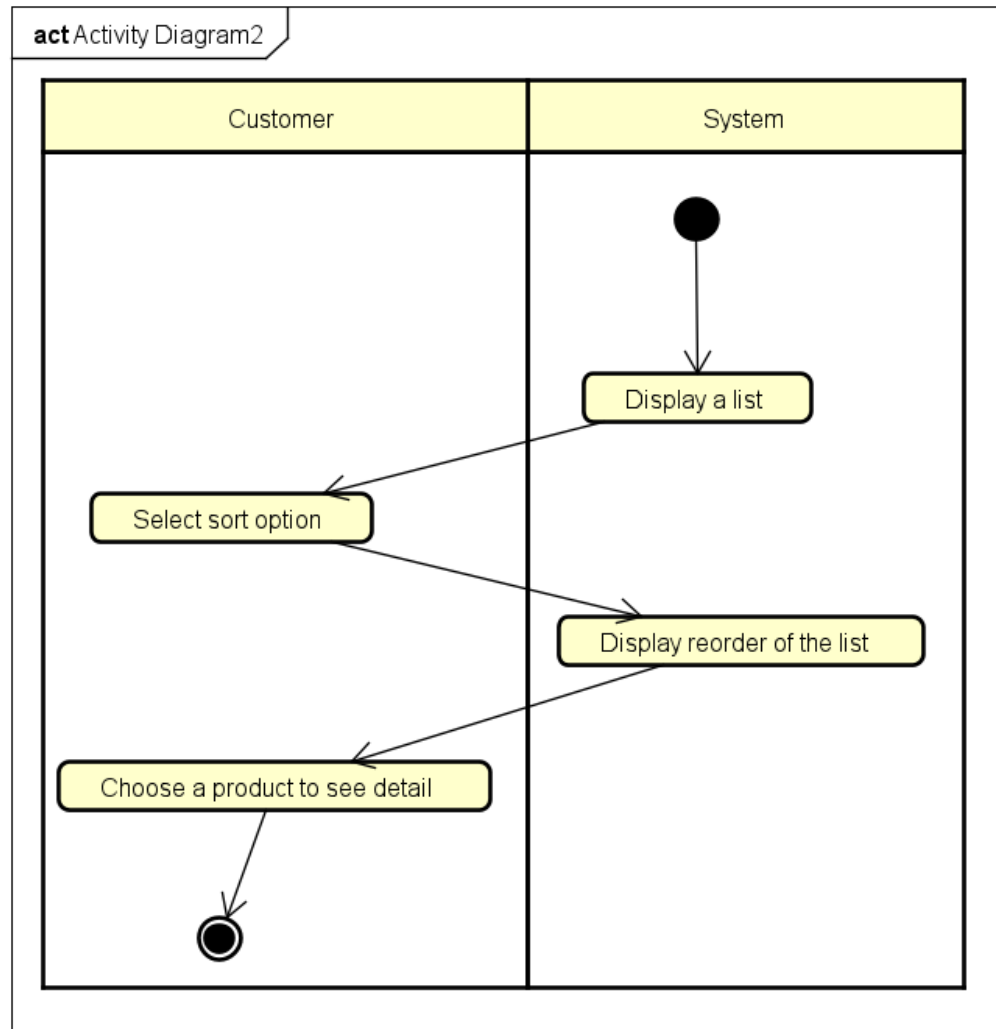
Activity Diagram cho UC “View List of Products”

2.3. Activity Diagram



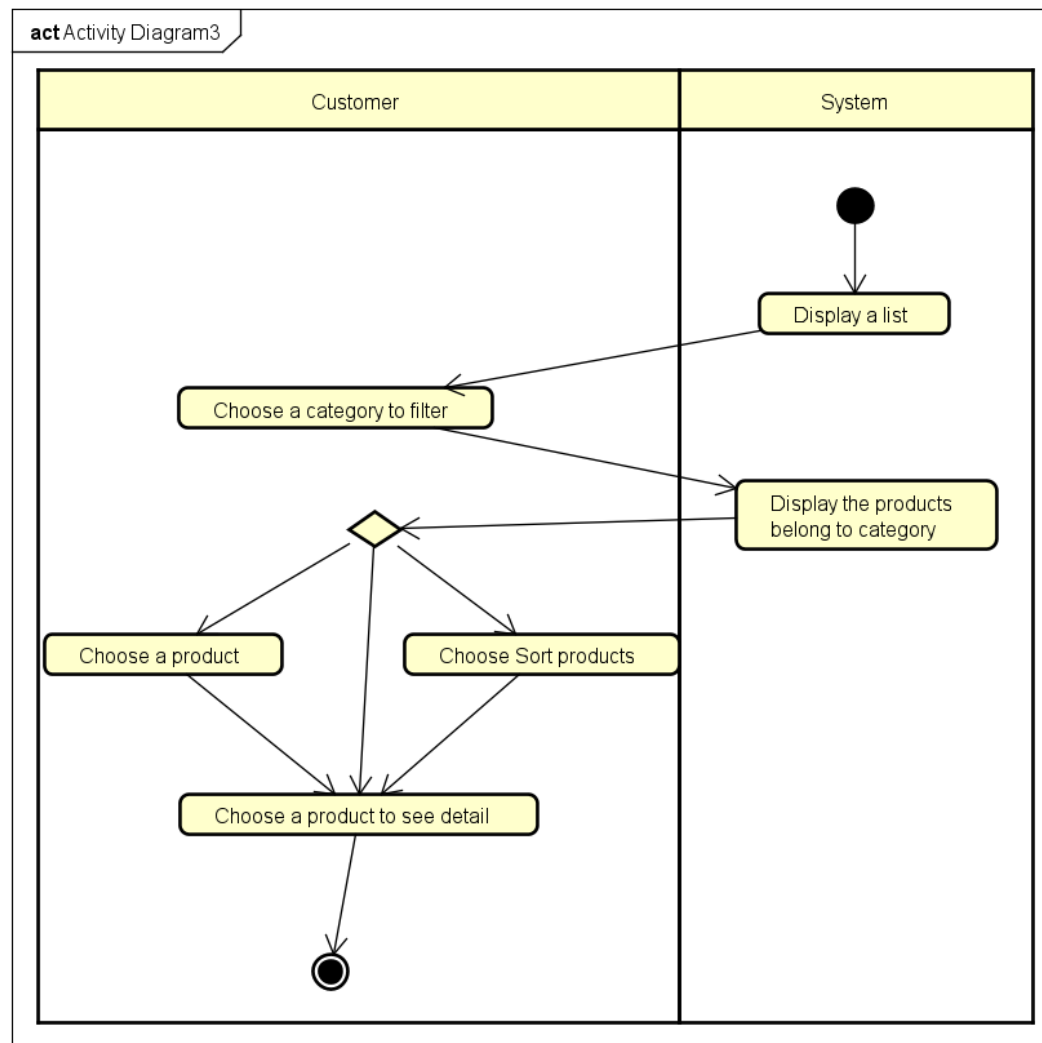
Activity Diagram cho UC “Search for products”

2.3. Activity Diagram



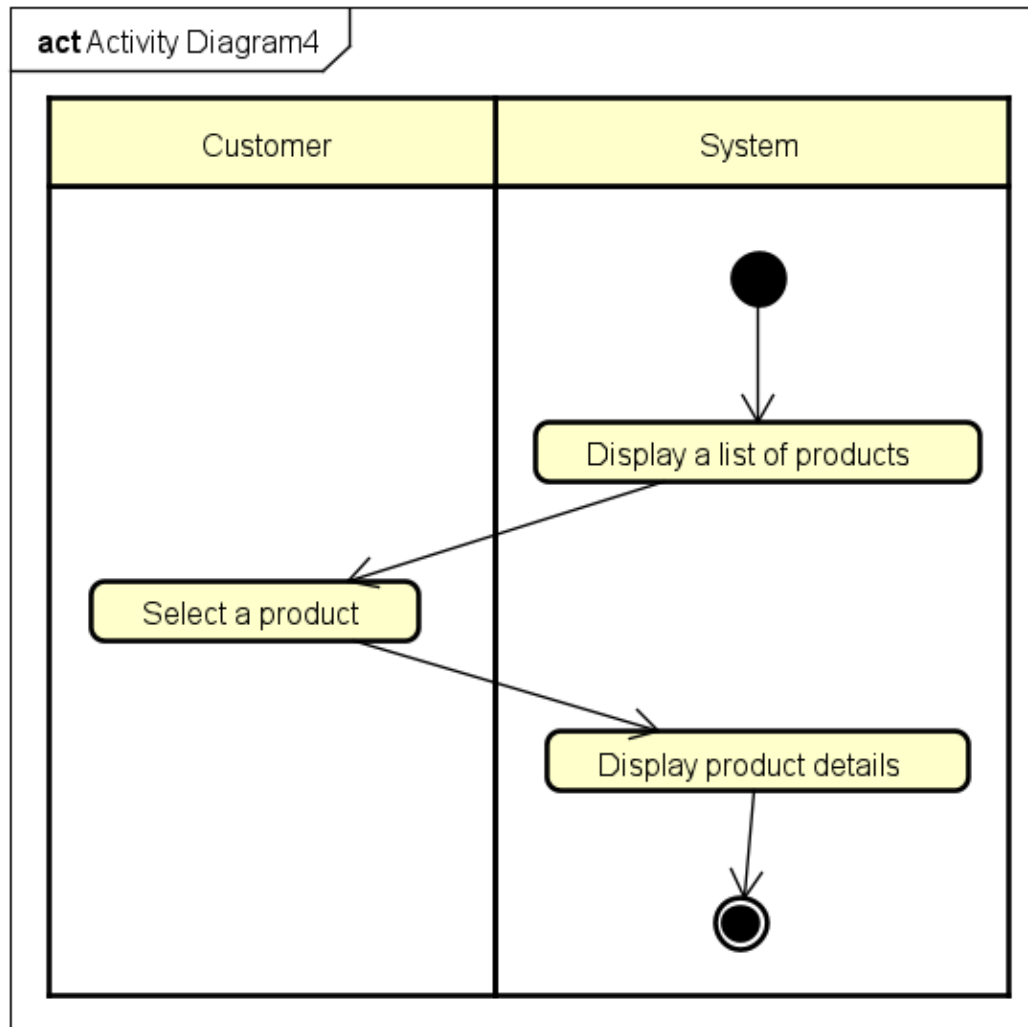
Activity Diagram cho UC "Sort products"

2.3. Activity Diagram



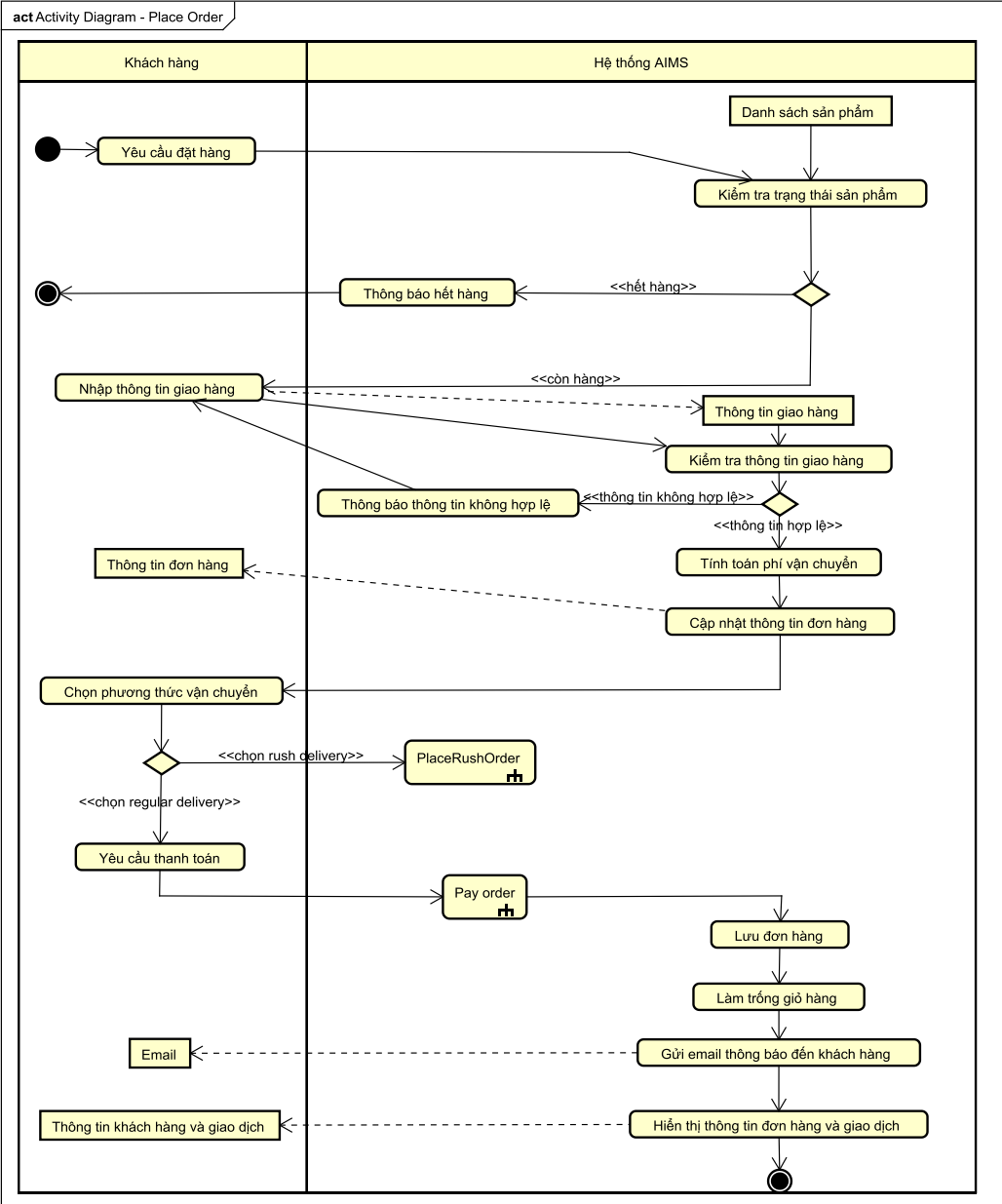
Activity Diagram cho UC “Filter by Category”

2.3. Activity Diagram

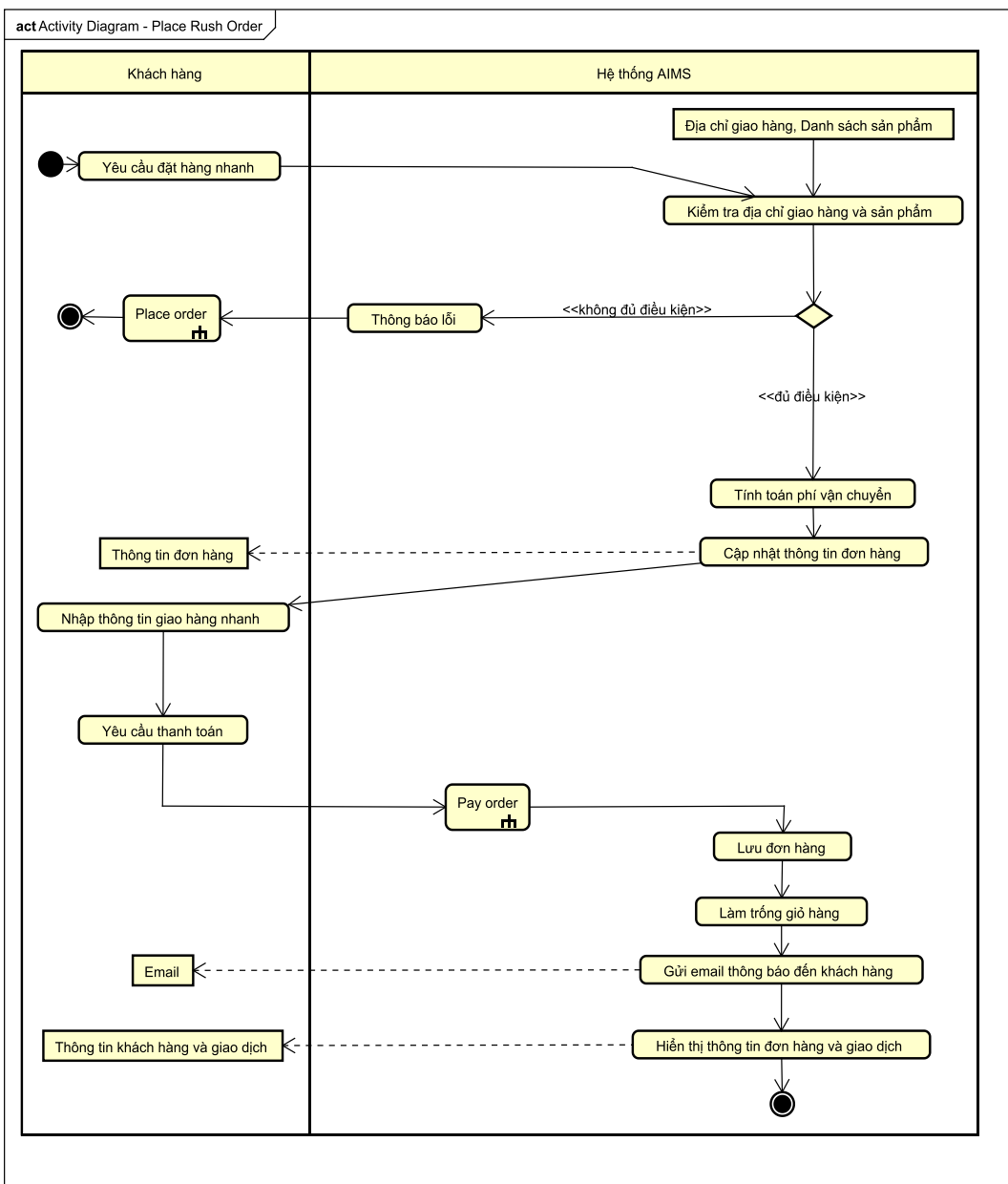


Activity Diagram cho UC “View Product Detail”

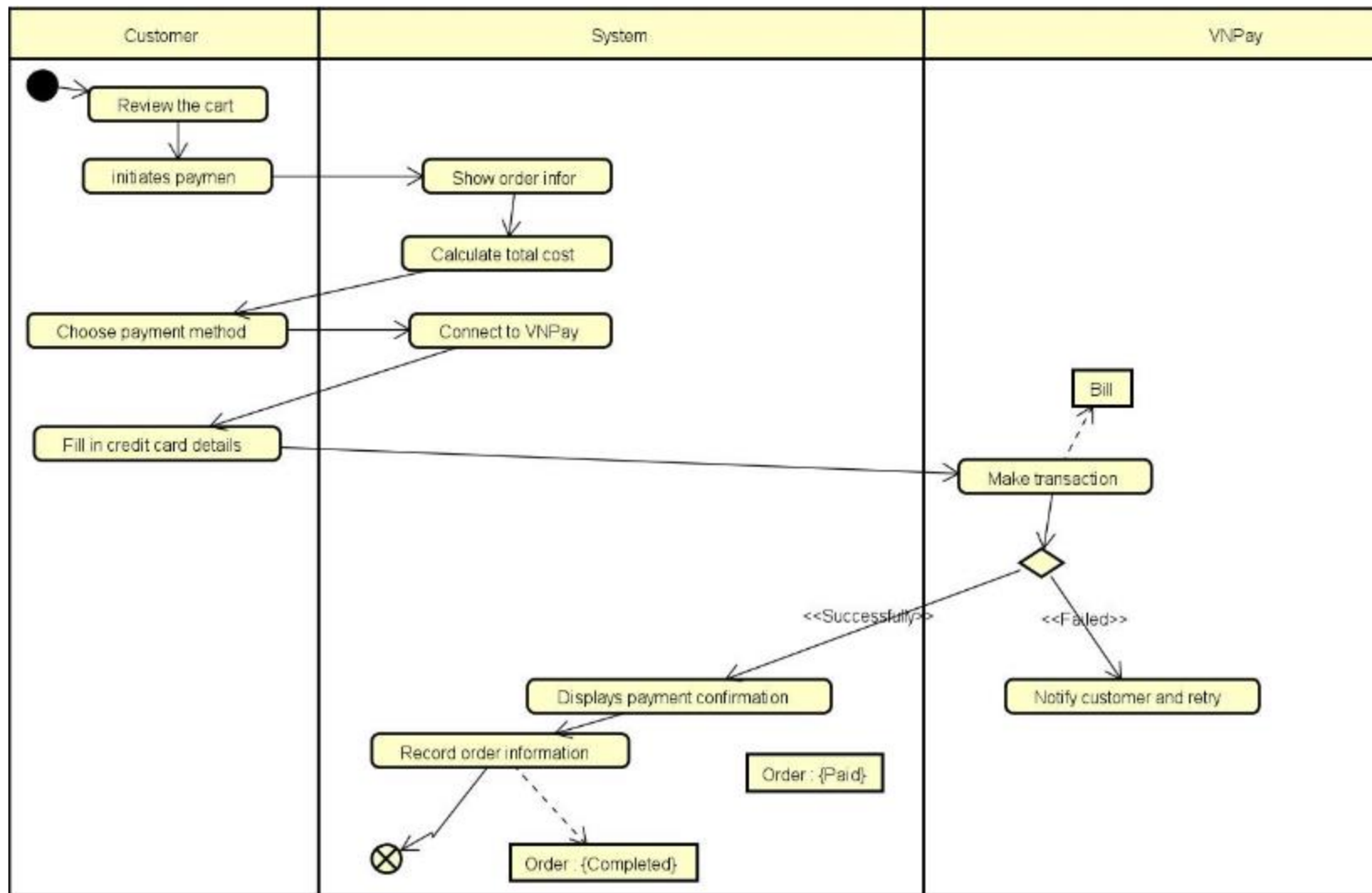
2.3. Activity Diagram



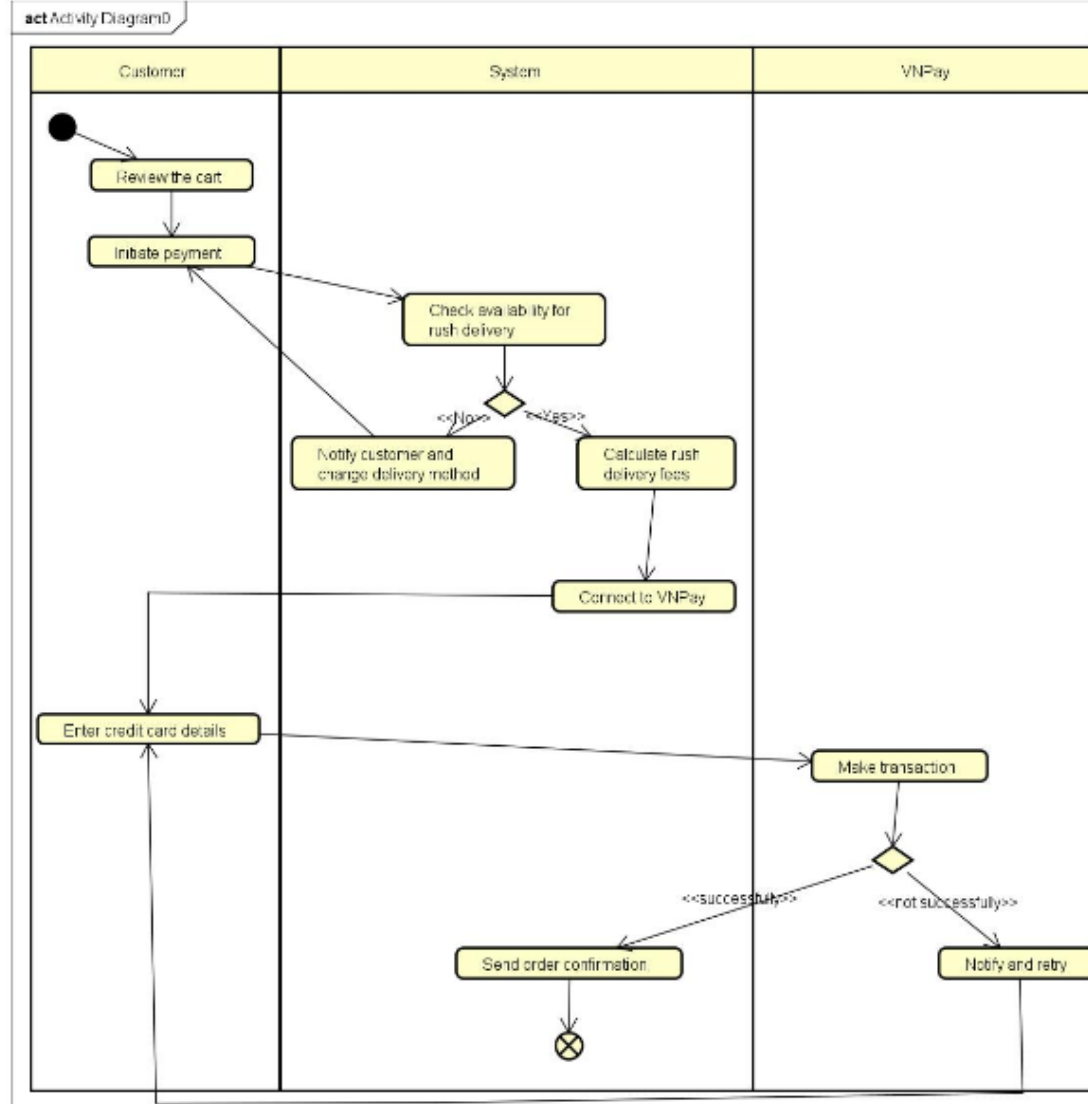
2.3. Activity Diagram



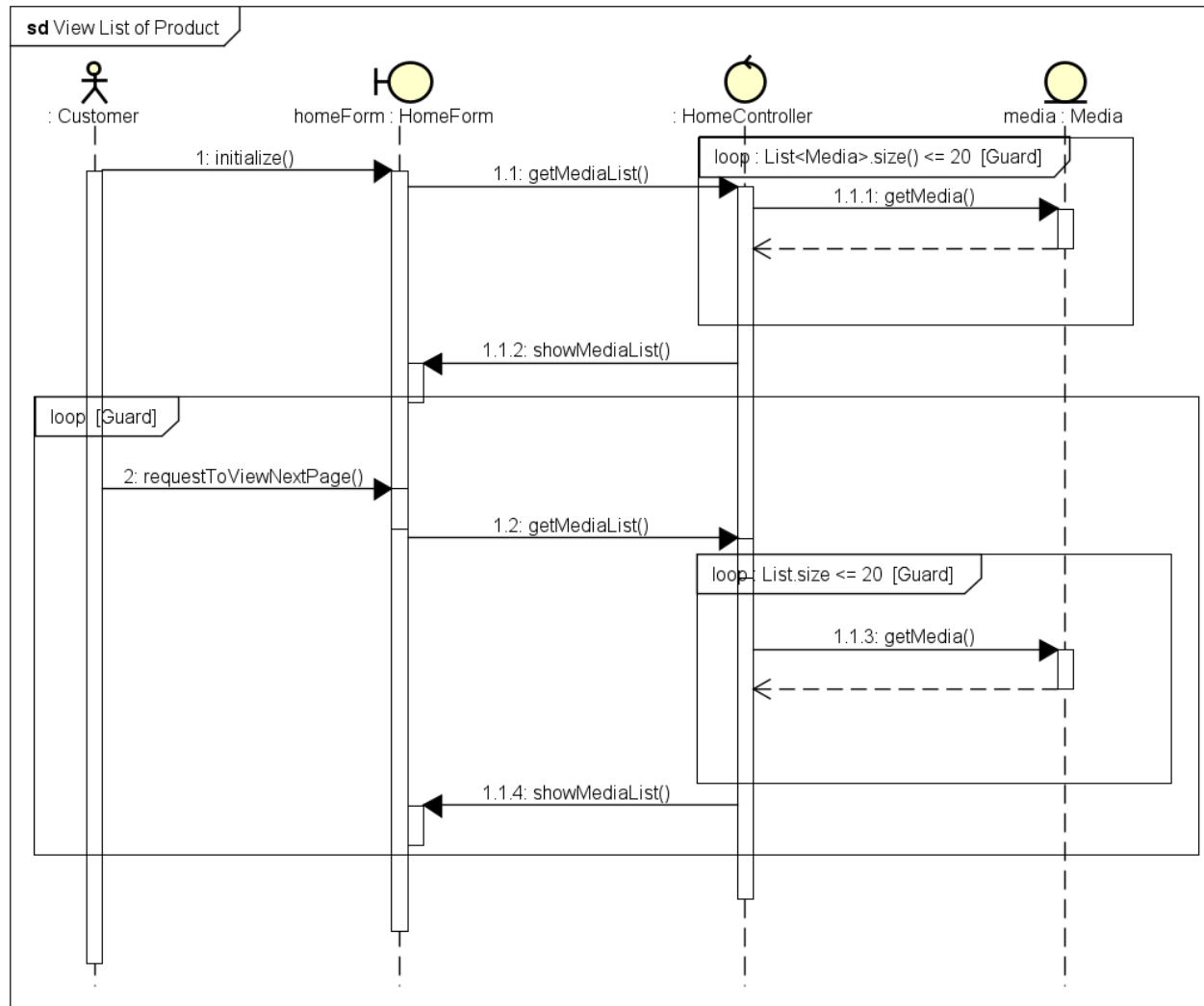
2.3. Activity Diagram



2.3. Activity Diagram

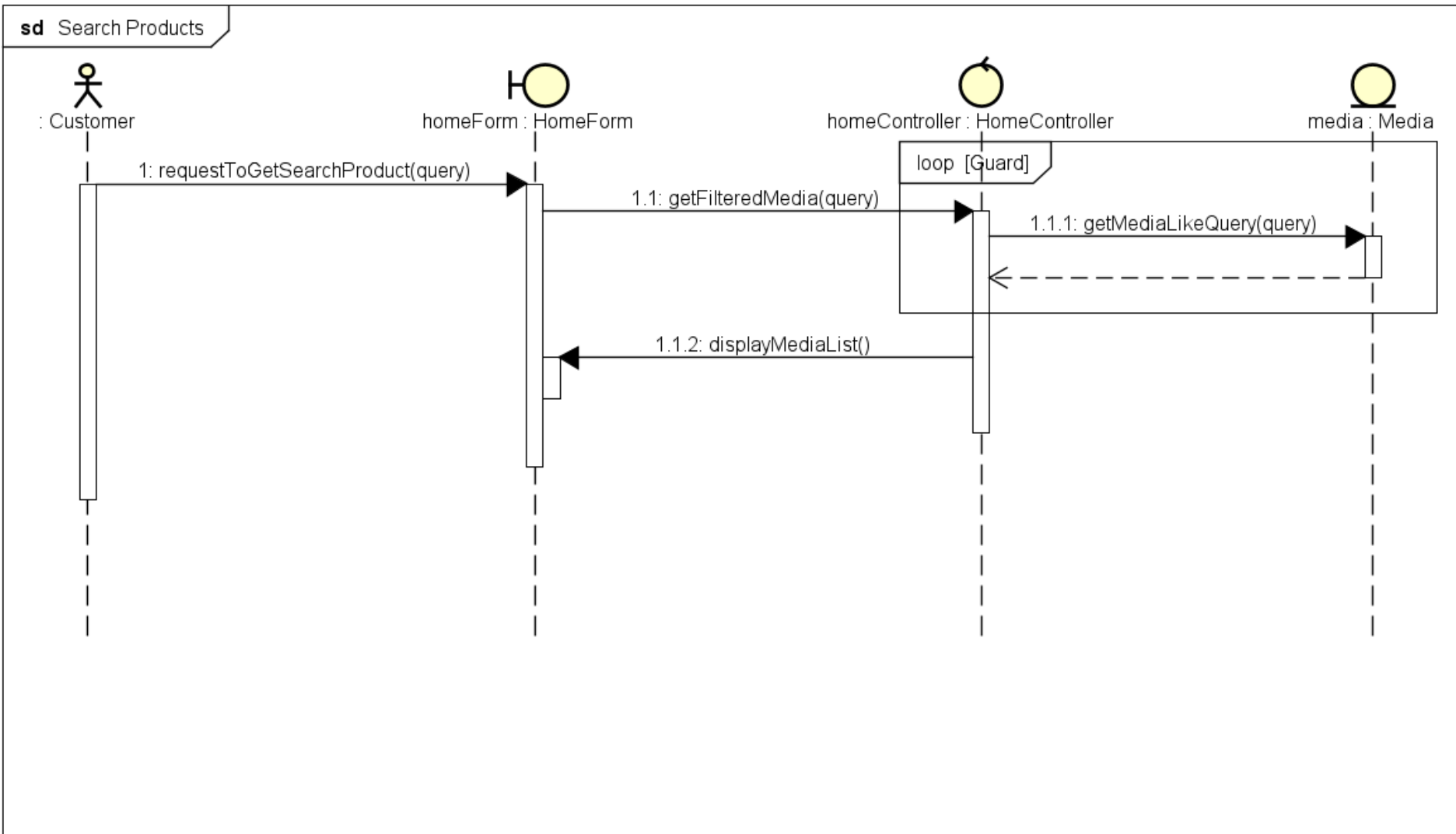


2.4. Sequence Diagram



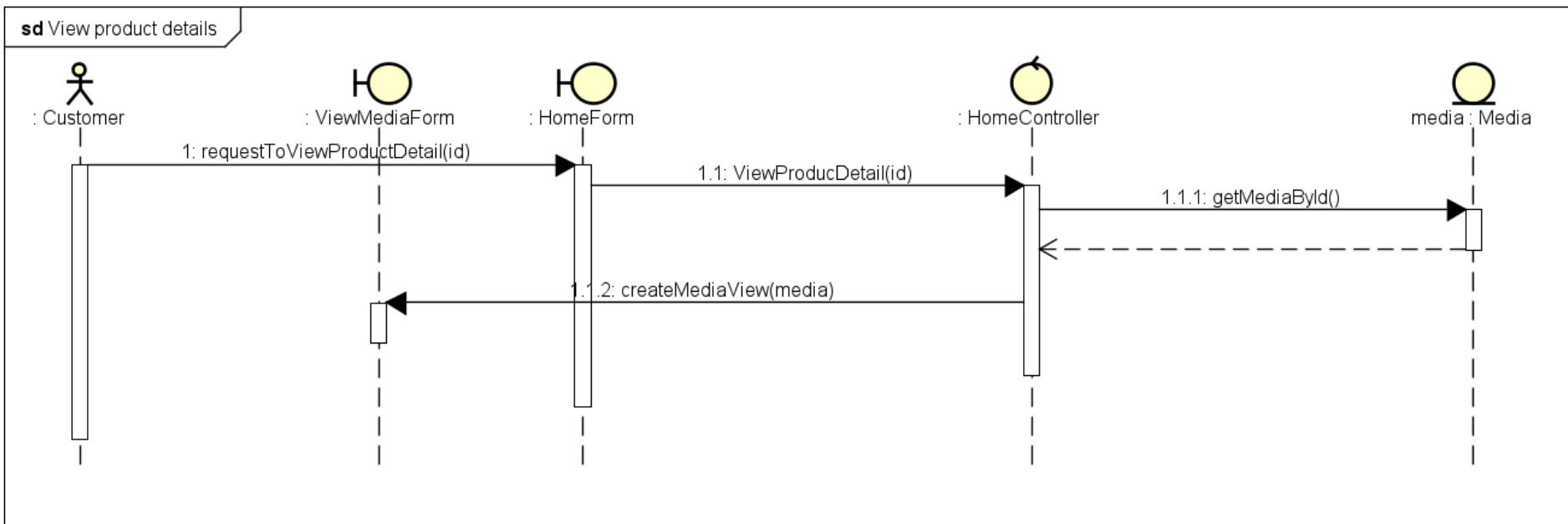
Sequence Diagram cho UC “View List of Product”

2.4. Sequence Diagram



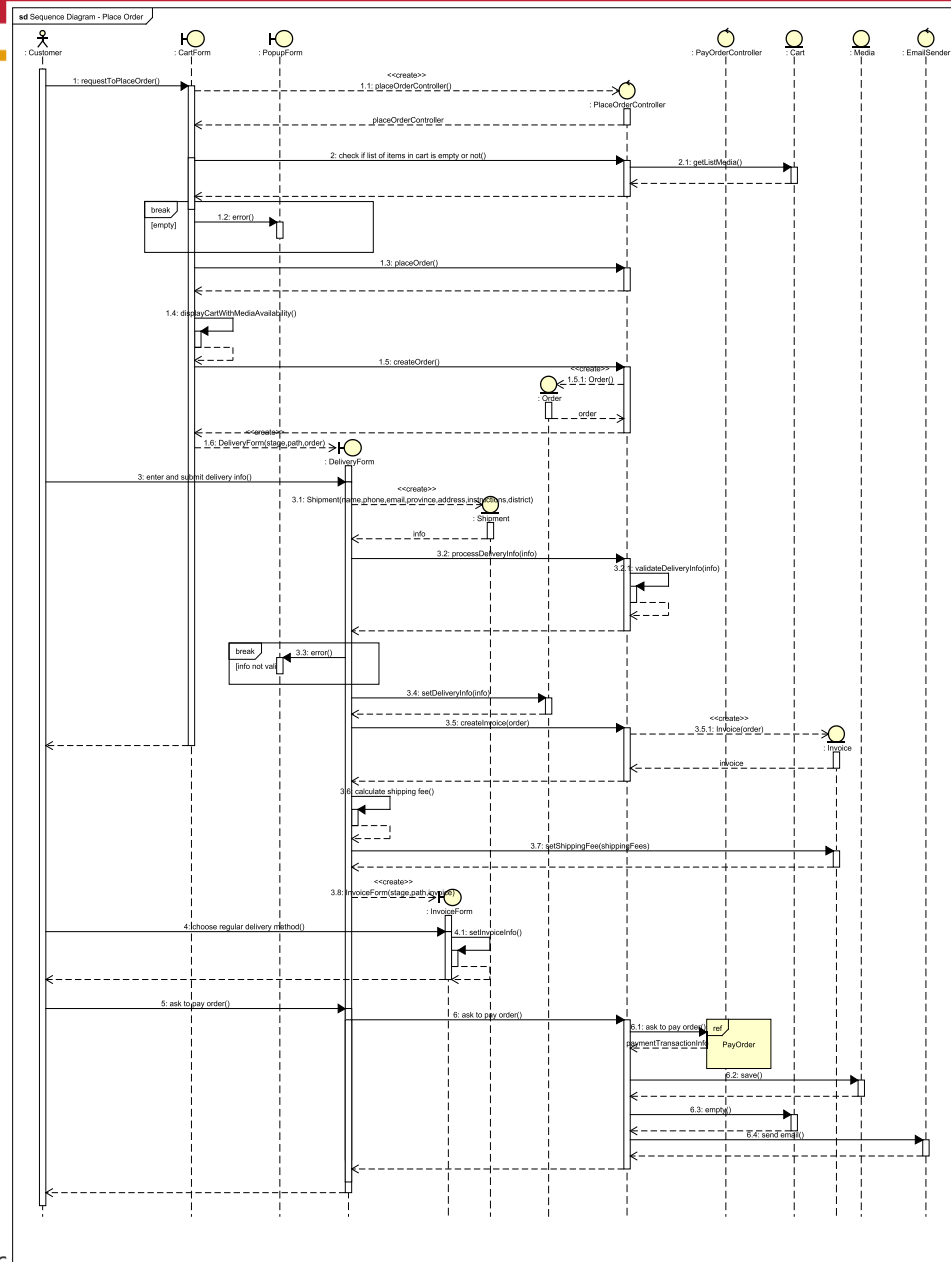
Sequence Diagram cho UC “Search Product”

2.4. Sequence Diagram

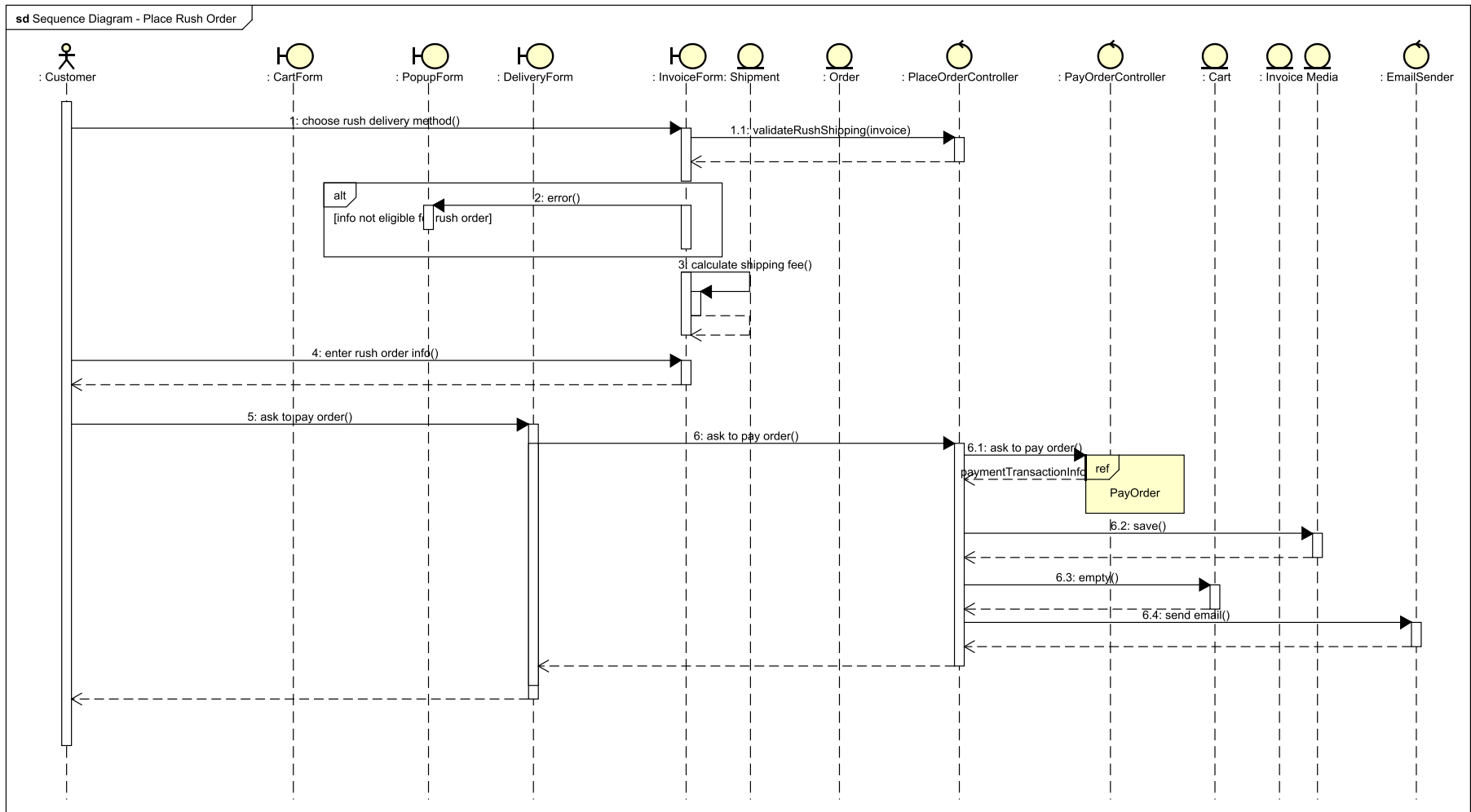


Sequence Diagram cho UC “View Product Detail”

2.4. Sequence Diagram

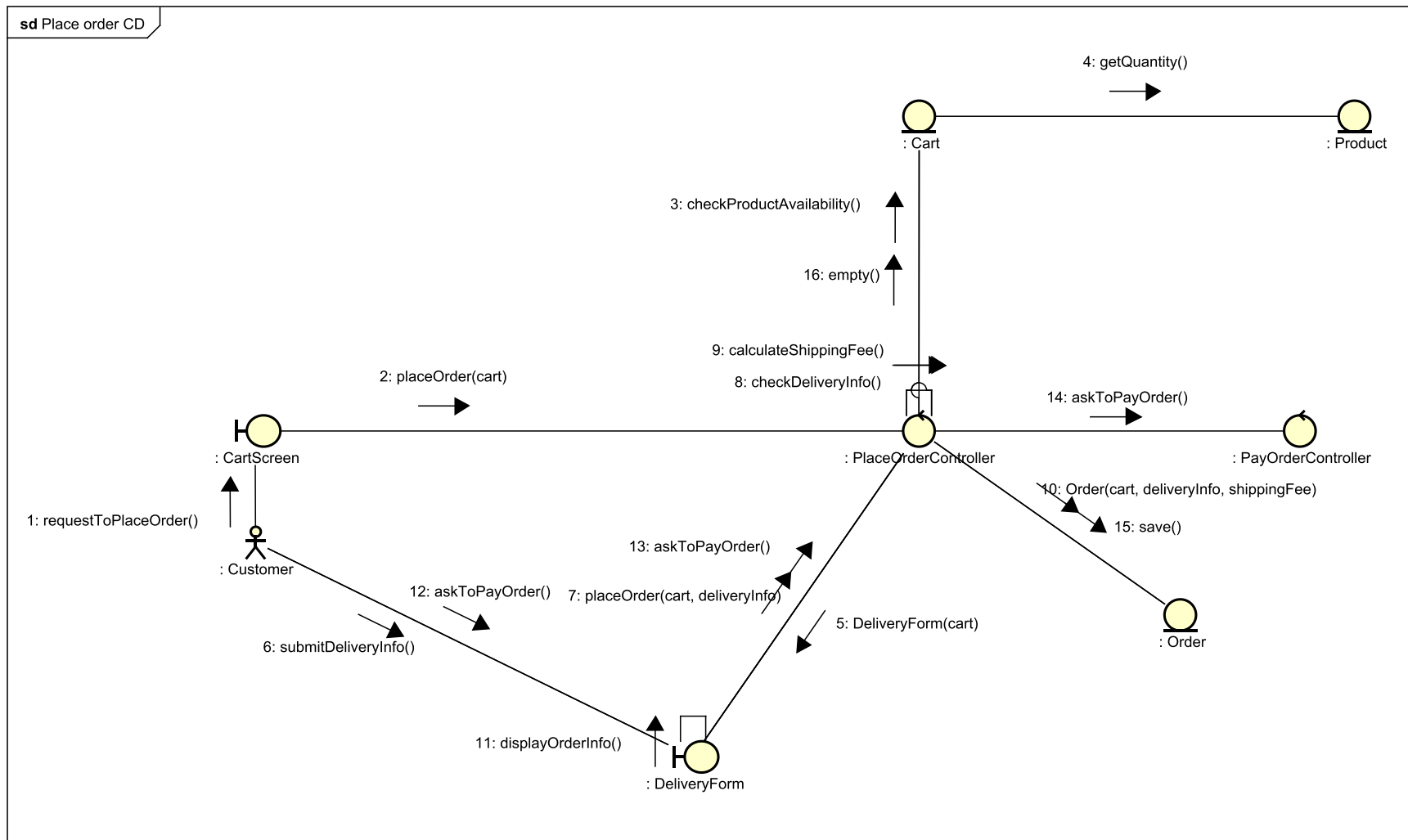


2.4. Sequence Diagram



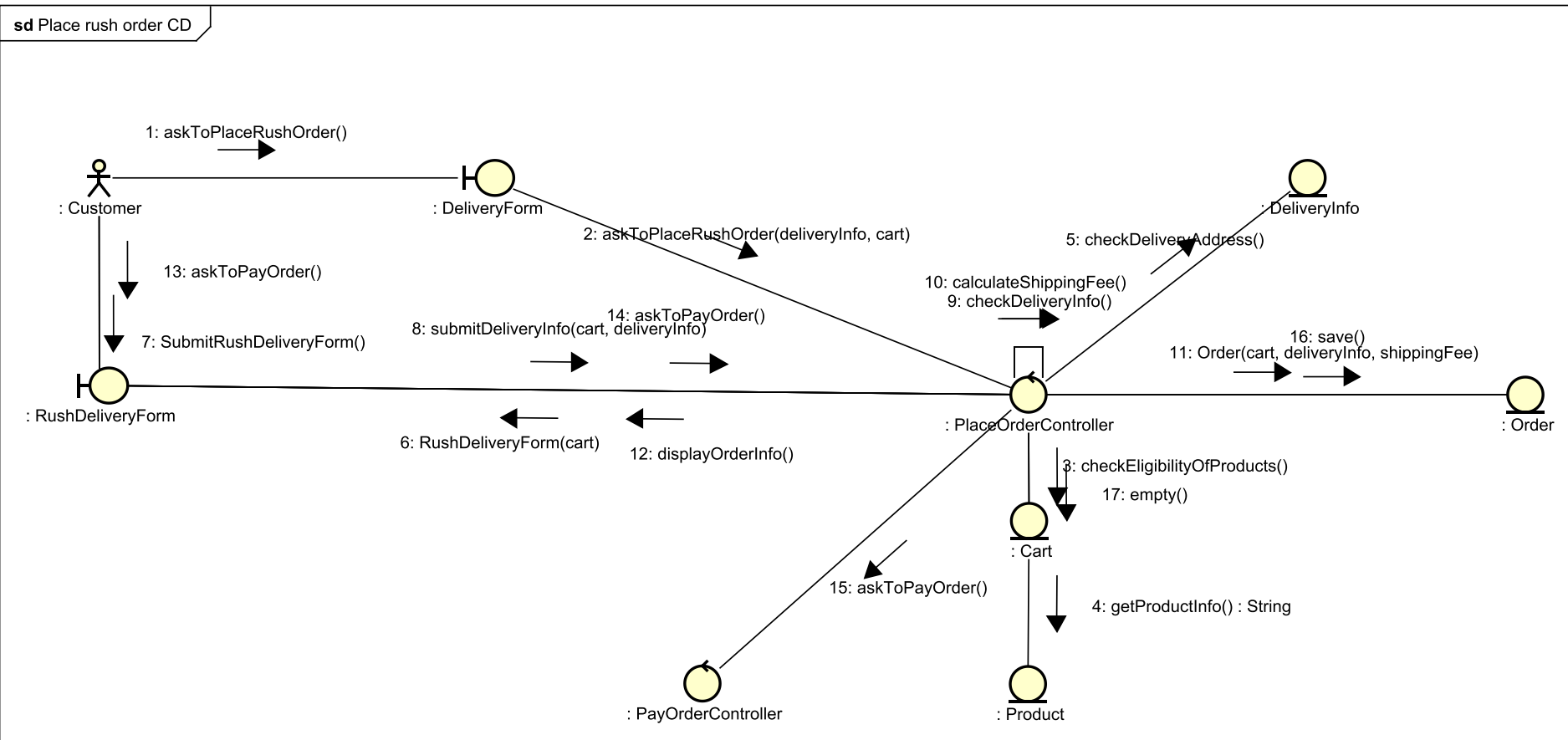
Sequence Diagram cho UC "Place rush order"

2.5. Communication Diagram



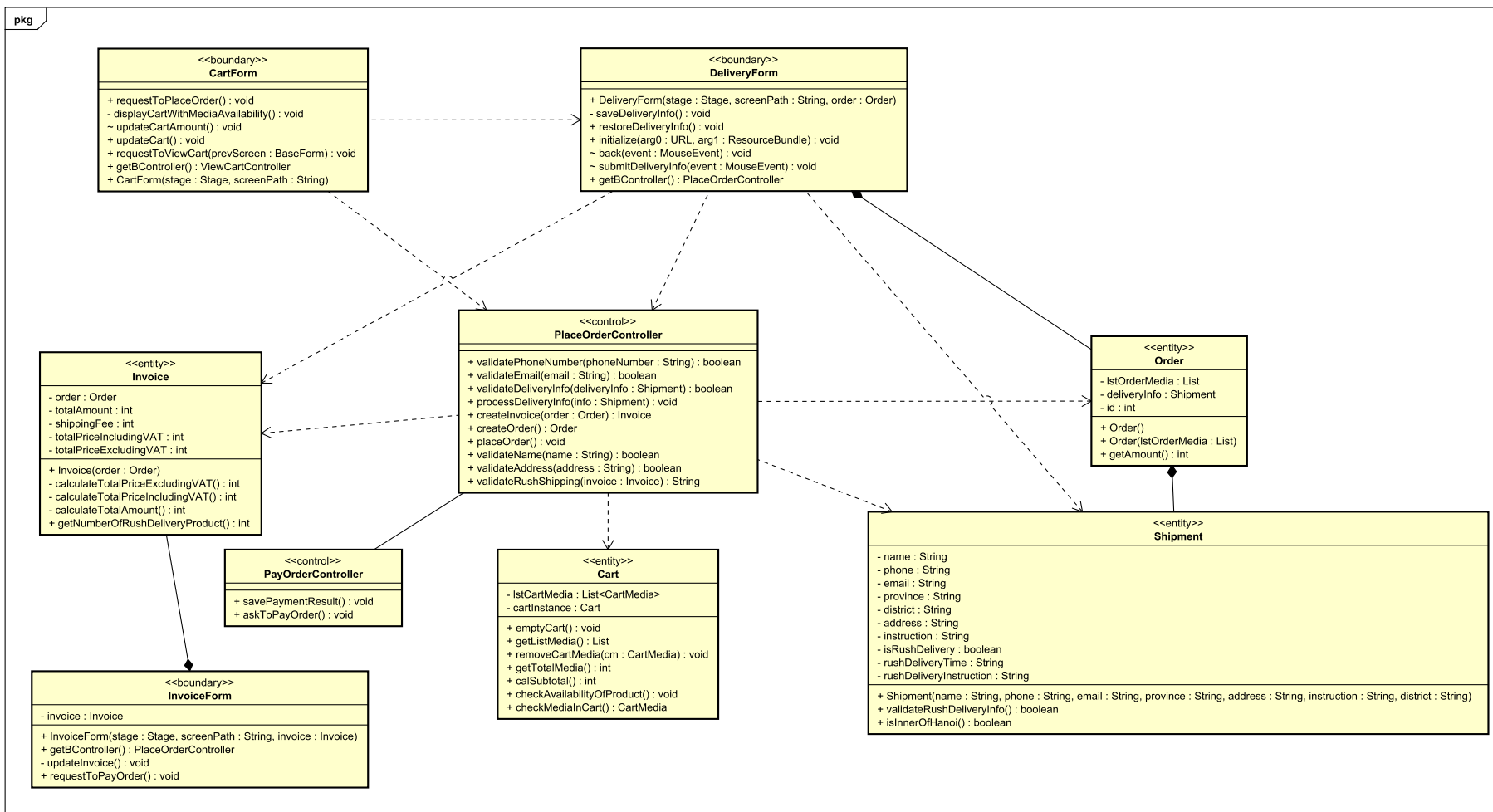
Communication Diagram cho UC "Place order"

2.5. Communication Diagram



Communication Diagram cho UC "Place rush order"

2.6. Class Diagram



Communication Diagram cho UC "Place order" và "Place rush order"

NỘI DUNG

3. Thiết kế chi tiết

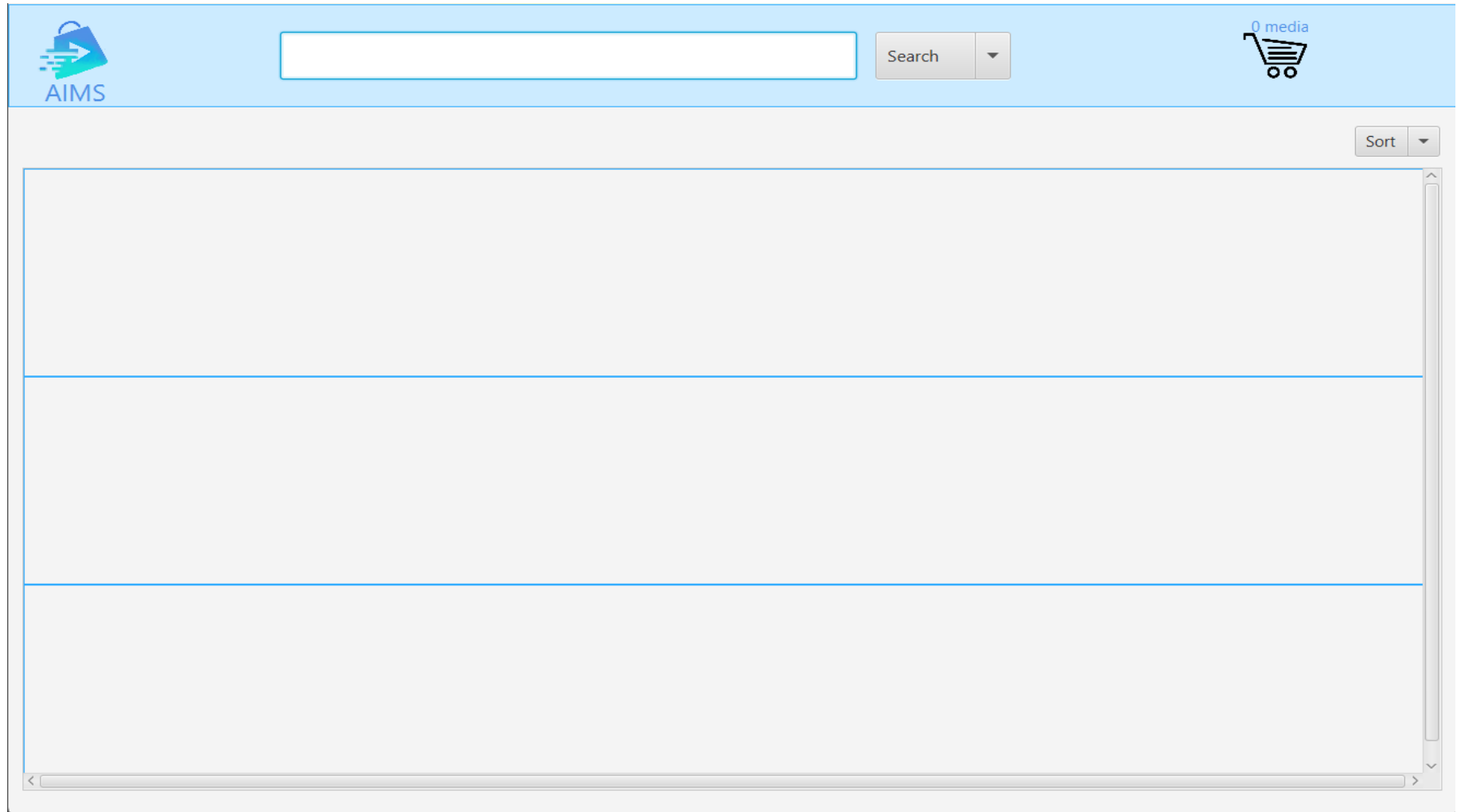
3.1. Thiết kế giao diện người dùng

3.2. Mô hình hóa dữ liệu

3.3.

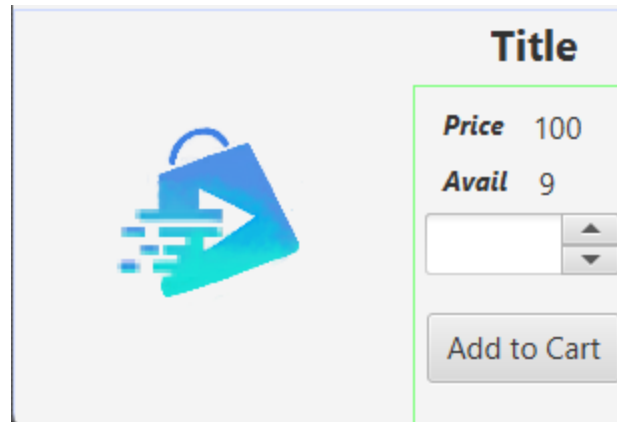
3.4. Thiết kế lớp

3.1. Thiết kế giao diện người dùng



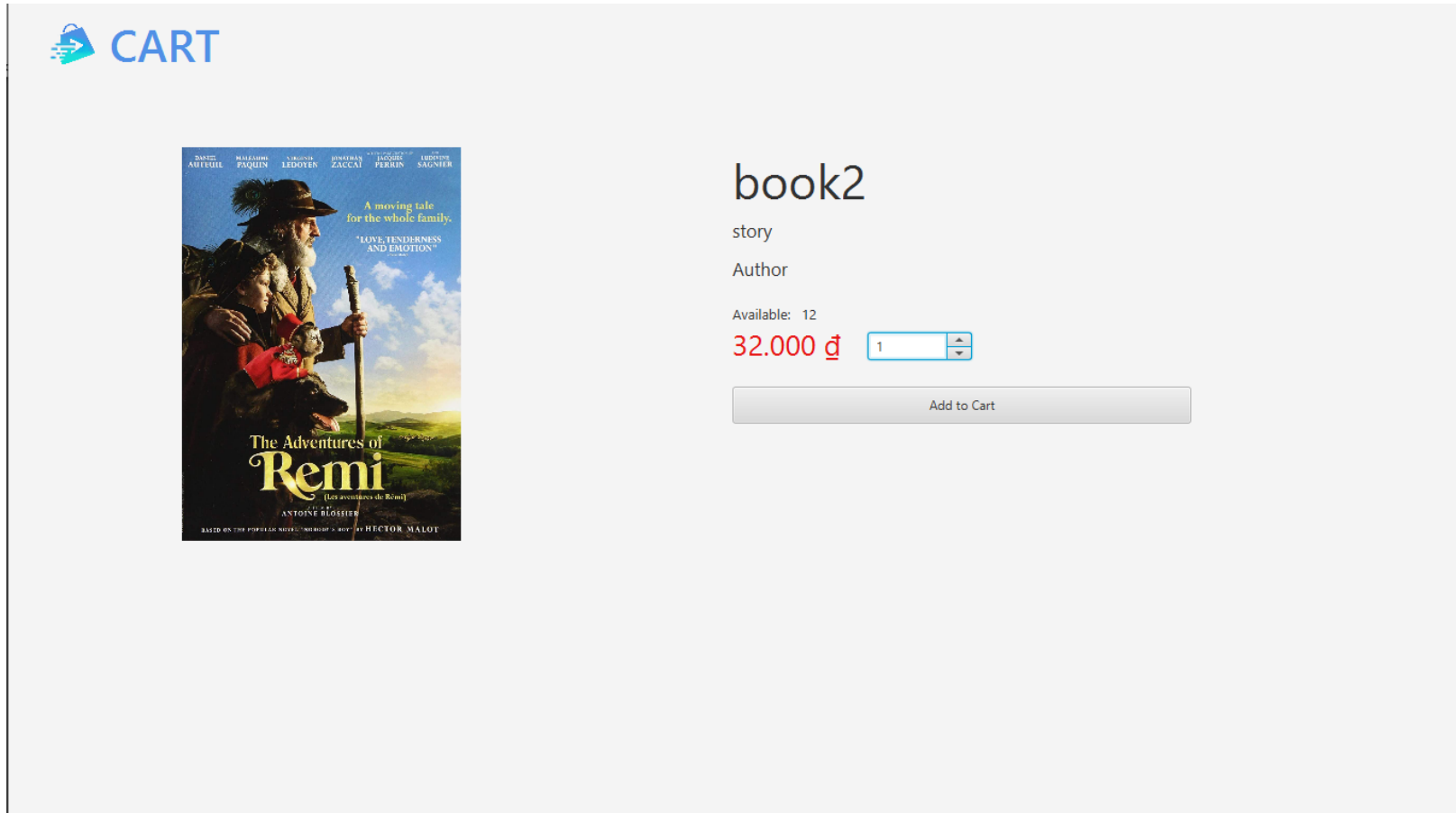
home.fxml

3.1. Thiết kế giao diện người dùng



media_home.fxml

3.1. Thiết kế giao diện người dùng



view_media.fxml

3.1. Thiết kế giao diện người dùng

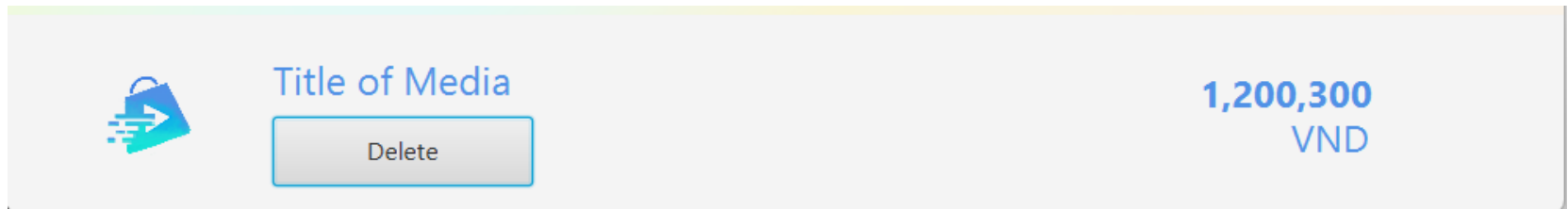
 **CART**

Subtotal: 0 VND
VAT (10%): 0 VND
Amount: 0 VND

Place order

cart.fxml

3.1. Thiết kế giao diện người dùng



media_cart.fxml

3.1. Thiết kế giao diện người dùng



*

Name

(a-zA-Z)

*

Phone

(0-9) 10 digits

*

Email

*

City

District

*

Address

(a-zA-Z)

Shipping Instructions

(a-zA-Z)

Back

Confirm delivery

shipping.fxml

3.1. Thiết kế giao diện người dùng

 **INVOICE**

Name

Full name

Phone

0987654321

Email

Email

City

Hanoi

Address

Address

Shipping Instructions

Shipping Instructions

Delivery Method

☒ Regular Delivery

☐ Rush Delivery

Delivery Time

Rush Delivery Instructions

Total Product Price (excluding VAT)

100,000 đ

Total Product Price (including VAT)

100,000 đ

Shipping Fees

50,000 đ

Total

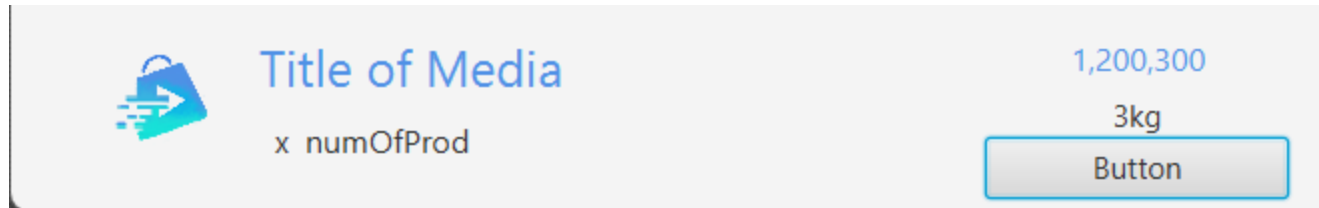
150,000 đ

Back

Confirm order

invoice.fxml

3.1. Thiết kế giao diện người dùng



media_invoice.fxml

NỘI DUNG

4. Phân tích thiết kế

4.1. Coupling và Cohesion

4.2. Nguyên tắc thiết kế

4.3. Design Patterns

4.1. Coupling và Cohesion

*Coupling:

- Vấn đề trong code mẫu: Liên quan đến 2 module Cart và Media: Toàn bộ object Media được truyền vào method `checkMediaInCart(Media media)` một cách không cần thiết, trong khi chỉ cần truyền media id
- Chỉ truyền dữ liệu cần thiết, cụ thể là id của Media

```
public CartMedia checkMediaInCart(Media media){  
    for (CartMedia cartMedia : lstCartMedia) {  
        if (cartMedia.getMedia().getId() == media.getId()) return cartMedia;  
    }  
    return null;  
}
```



```
1 usage  minhtruong171123 *  
public CartMedia checkMediaInCart(int mediaId){  
    for (CartMedia cartMedia : lstCartMedia) {  
        if (cartMedia.getMedia().getId() == mediaId) return cartMedia;  
    }  
    return null;  
}
```

4.1. Coupling và Cohesion

*Cohesion:

- Vấn đề trong code mẫu: Trong module Configs chứa các thành phần cấu hình của dự án AIMS, bao gồm các URL API, cấu hình cơ sở dữ liệu, dữ liệu demo, đường dẫn tệp tin tài nguyên tĩnh, và thông tin liên quan đến hệ thống như tỷ lệ VAT và danh sách các tỉnh.

Giải pháp: Tách thành các lớp/module riêng biệt: Có thể chia lớp Configs thành nhiều lớp con hoặc module khác nhau, mỗi lớp/module đảm nhiệm một phần cấu hình cụ thể, ví dụ:

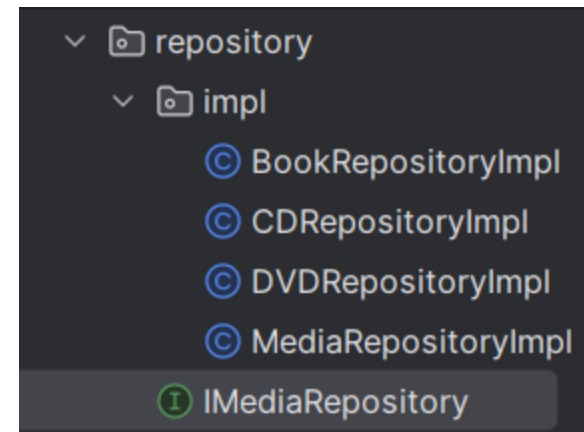
- + APIConfigs: Chứa các URL API.
- + DatabaseConfigs: Chứa các cấu hình cơ sở dữ liệu.
- + StaticResourcesConfigs: Chứa các đường dẫn tệp tin tài nguyên tĩnh (hình ảnh, FXML).
- + GlobalSettingsConfigs: Chứa các cài đặt toàn cục như tỷ lệ VAT và loại tiền tệ.
- + ProvincesConfigs: Chứa danh sách các tỉnh.

4.2. Nguyên tắc thiết kế

Open-Closed Principle:

- Vấn đề ở thiết kế cũ: Đối với các lớp Media, Book, CD, DVD, phương thức `getMediaById` và `getAllMedia` chứa logic truy vấn SQL trực tiếp trong lớp, khiến việc mở rộng cách thức truy vấn hoặc thay đổi cấu trúc dữ liệu khó khăn.
- Giải pháp: Trích xuất logic truy vấn cơ sở dữ liệu ra một lớp Repository, tách biệt logic thao tác dữ liệu với logic nghiệp vụ hoạt động trên model

```
17 public class Media {  
52     public Media getMediaById(int id) throws SQLException{  
53         String sql = "SELECT * FROM Media ;";  
54         Statement stm = DBConnection.getConnection().createStatement();  
55         ResultSet res = stm.executeQuery(sql);  
56         if(res.next()) {  
57             return new Media()  
58                 .setId(res.getInt("id"))  
59                 .setTitle(res.getString("title"))  
60                 .setQuantity(res.getInt("quantity"))  
61                 .setCategory(res.getString("category"))  
62                 .setMediaURL(res.getString("imageUrl"))  
63                 .setPrice(res.getInt("price"))  
64                 .setType(res.getString("type"));  
65         }  
66         return null;  
67     }  
68 }  
69  
70 public List getAllMedia() throws SQLException{  
71     Statement stm = DBConnection.getConnection().createStatement();  
72     ResultSet res = stm.executeQuery("select * from Media");  
73     ArrayList medium = new ArrayList<>();  
74     while (res.next()) {  
75         Media media = new Media()  
76             .setId(res.getInt("id"))  
77             .setTitle(res.getString("title"))  
78             .setQuantity(res.getInt("quantity"))  
79             .setCategory(res.getString("category"))  
80             .setMediaURL(res.getString("imageUrl"))  
81             .setPrice(res.getInt("price"))  
82             .setType(res.getString("type"));  
83         medium.add(media);  
84     }  
85     return medium;  
86 }
```



4.2. Nguyên tắc thiết kế

Open-Closed Principle:

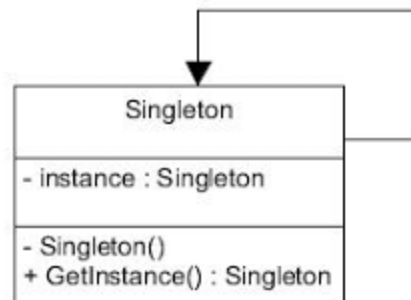
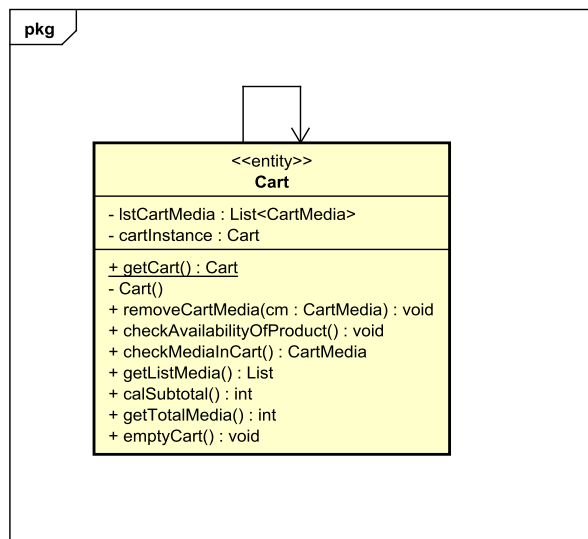
- Vấn đề ở thiết kế cũ: Đối với các lớp Media, Book, CD, DVD, phương thức `getMediaById` và `getAllMedia` chứa logic truy vấn SQL trực tiếp trong lớp, khiến việc mở rộng cách thức truy vấn hoặc thay đổi cấu trúc dữ liệu khó khăn.
- Giải pháp: Trích xuất logic truy vấn cơ sở dữ liệu ra một lớp Repository, tách biệt logic thao tác dữ liệu với logic nghiệp vụ hoạt động trên model

```
8 usages 4 implementations 1 Truong Giang
public interface IMediaRepository<T> {
    1 usage 4 implementations 1 Truong Giang
    T getById(int id) throws SQLException;
    4 implementations 1 Truong Giang
    List<T> getAll() throws SQLException;
}
```

```
13 public class MediaRepositoryImpl implements IMediaRepository {
14     3 usages
15     private final Statement stm;
16
17     2 usages 1 Truong Giang
18     public MediaRepositoryImpl() throws SQLException {
19         stm = DBConnection.getConnection().createStatement();
20     }
21
22     1 Truong Giang +1
23     @Override
24     public List<Media> getAll() throws SQLException {
25         String sql = "SELECT * FROM Media";
26         List<Media> medium = new ArrayList<>();
27         ResultSet res = stm.executeQuery(sql);
28         while (res.next()) {
29             medium.add(mapToMedia(res));
30         }
31         System.out.println(medium);
32         return medium;
33     }
34
35     no usages 1 Truong Giang
36     public void updateMediaFieldById(int id, String field, Object value) throws SQLException {
37         if (value instanceof String) {
38             value = "\"" + value + "\"";
39         }
40         String sql = "UPDATE Media SET " + field + " = " + value + " WHERE id = " + id + ";";
41         stm.executeUpdate(sql);
42     }
43 }
```

4.3. Design Patterns

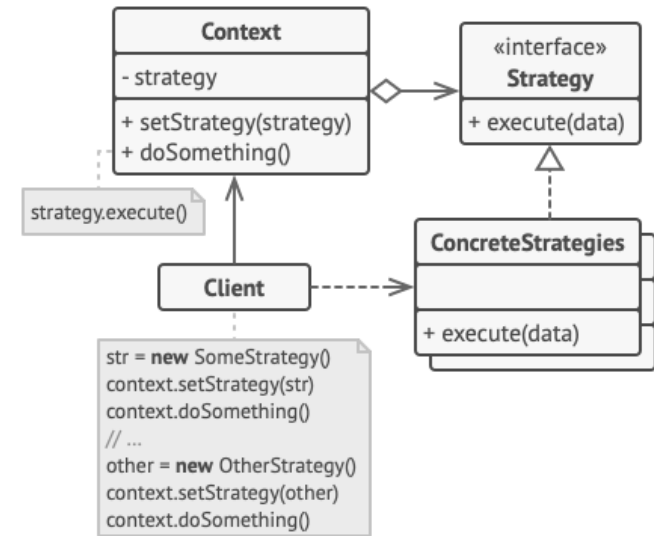
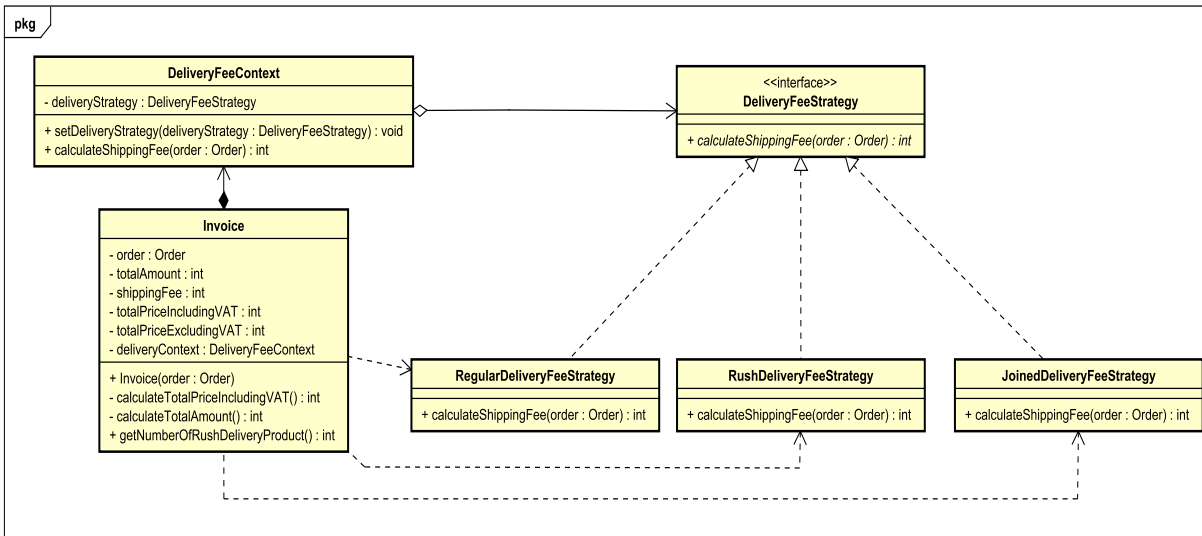
***Singleton Design Pattern:** đảm bảo chỉ có duy nhất 1 instance của Cart trong ứng dụng, giúp quản lý trạng thái của giỏ hàng trong toàn bộ ứng dụng



- **cartInstance:** Dữ liệu thành viên instance (private và static) là đối tượng duy nhất của lớp Singleton.
- **Cart():** Constructor của lớp Singleton được định nghĩa thành private để người dùng không thể tạo thực thể trực tiếp từ bên ngoài lớp.
- **getCart():** dùng để khởi tạo đối tượng duy nhất, định nghĩa thành public và static. Client chỉ dùng `GetInstance()` để tạo đối tượng cho lớp Singleton.
- Thực hiện khởi tạo chậm (lazy initialization) trong `getCart()`: chỉ khi gọi phương thức `getCart()` mới khởi tạo đối tượng Cart. Phương thức này trả về một thể hiện mới hay null tùy thuộc vào một tham số kiểu boolean dùng như cờ hiệu báo xem lớp Singleton đã tạo thể hiện hay chưa.

4.3. Design Patterns

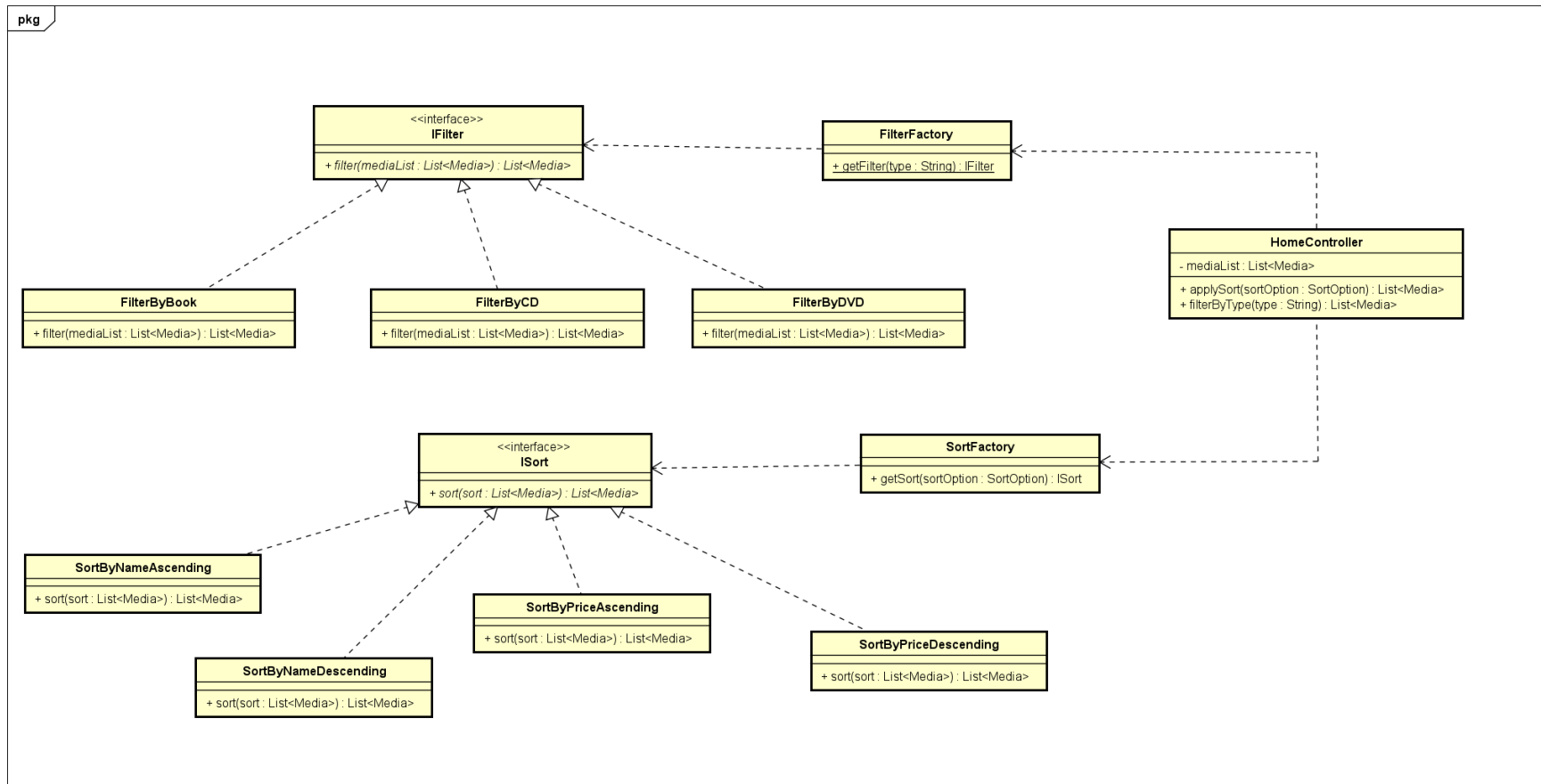
***Strategy Design Pattern:** giúp chuyển đổi linh hoạt giữa các thuật toán tính phí vận chuyển khác nhau, để mở rộng khi có thêm cách tính phí vận chuyển mới trong tương lai



- **Context (DeliveryFeeContext)**: duy trì một tham chiếu đến một trong các strategy cụ thể và giao tiếp với các đối tượng này thông qua interface Strategy. Context để lộ một setter cho client thay thế với strategy được liên kết với context khi đang chạy.
- **Strategy (DeliveryFeeStrategy)**: cung cấp một interface chung cho context giao tiếp với các strategy object
- **Concrete Strategy (RegularDeliveryFeeStrategy, RushDeliveryFeeStrategy, JoinedDeliveryFeeStrategy)**: implement các thuật toán khác nhau cho context sử dụng
- **Client (Invoice)**: có trách nhiệm tạo ra các strategy object và truyền vào cho context sử dụng

4.3. Design Patterns

***Factory Design Pattern:** Khi thêm một loại hàng mới (như hat, pen, notebook,...) chỉ cần thực thi lớp Interface và thêm tùy chọn ở trong Factory class mà không cần phải sửa đổi mã nguồn.



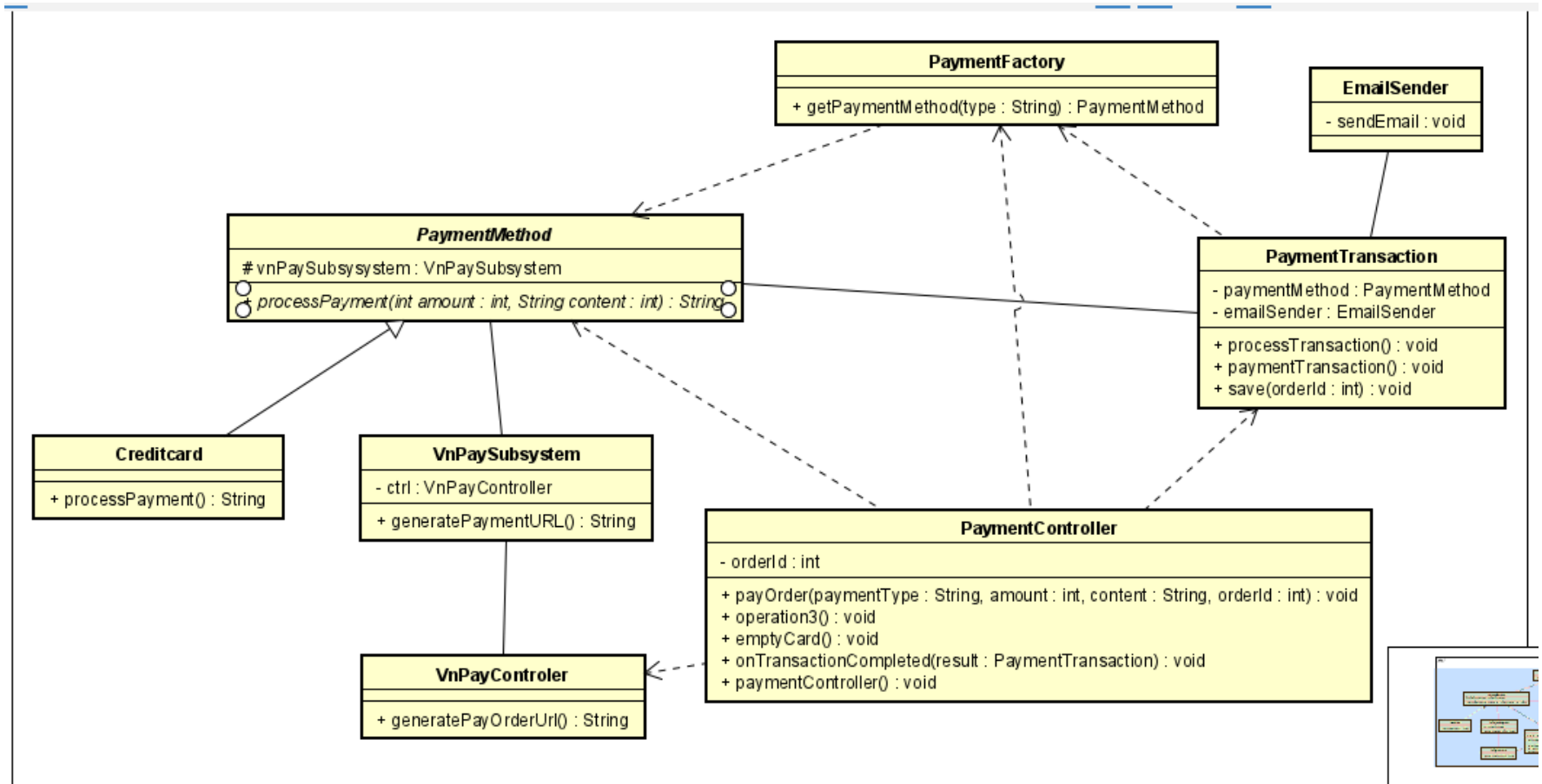
4.4. Design Patterns

- **Vấn đề:** Các lớp chịu trách nhiệm xử lý logic thanh toán sẽ phải chứa cả logic khởi tạo đối tượng và logic nghiệp vụ chính với các câu điều kiện để chọn loại thẻ nếu mở rộng nhiều loại thẻ trong tương lai.
- **Control Coupling:** có nhiều lệnh điều kiện .
- **SRP:** Thêm nhiệm vụ tạo đối tượng Card mới.
- **Open-Closed Principle:** phải sửa mã hiện có để thêm tính năng mới.
- **DIP :** PaymentTranction phụ thuộc trực tiếp vào CreditCard .

4.4 Design Patterns

- **Open/Closed Principle (OCP):** Thêm phương thức thanh toán mới chỉ cần sửa Factory, không phải sửa các logic hiện có.
- **Single Responsibility Principle (SRP):** Tách biệt logic khởi tạo đối tượng và logic xử lý nghiệp vụ.
- **Dependency Inversion Principle (DIP):** Các lớp logic chính phụ thuộc vào abstraction (PaymentMethod), không phụ thuộc lớp cụ thể.

4.3. Design Patterns



A large, stylized graphic on the left side of the slide. It consists of a red background with a circular pattern of white dots of varying sizes, creating a sense of depth and movement. The word "HUST" is written in white, bold, sans-serif capital letters in the center of this graphic.

HUST

THANK YOU !

A large, stylized graphic of the HUST logo, consisting of a circular arrangement of dots in a halftone pattern, centered on a red background.

HUST